

Multiagent Information Coverage Inspired Target Searching and Tracking in Stone Soup

Timothy J. Glover^{*†}, James Knowles^{*}, Henry Pritchett[†], Matthew Coombes^{*}

Abstract—Searching and tracking for mobile targets with multiple mobile sensing platforms is a relevant capability for many real-world scenarios such as search and rescue and surveillance. This paper presents an approach to performing searching and tracking for a single target of interest using multiple mobile sensor systems in urban environments. Formulating the searching and tracking problem as a partially observable Markov decision process, a coverage control objective inspired reward function is designed, to create a single objective function for both searching and tracking behaviours. Furthermore, the approach alleviates differentiability requirements in typical coverage control approaches. Visibility informed Bernoulli filter provides a suitable tracking algorithm estimating both probability that a target exists and its state. Road network information is used to inform the information coverage objective for scenario tailored search behaviour. Monte Carlo tree search is adopted to implement the algorithm non-myopically and balance exploitation of immediate rewards with exploration of potential future gains. Contribution to the open source Stone Soup framework creates several new components for the toolkit, where the flexibility of the approach can be leveraged with the established wealth of tracking and filtering algorithms. Simulated Urban Mobility software package is integrated with Stone Soup to create a realistic urban simulation scenario with two sensing agents. Through Monte Carlo simulations, the algorithm demonstrates efficient search behaviour, good target tracking performance and effective coverage of the target once found.

I. INTRODUCTION

Managing sensors to search for and track objects of interest is a problem applicable to numerous fields including search and rescue [1]–[5], wildlife monitoring [6]–[8] and surveillance [9], [10]. Effectively leveraging controllable degrees of freedom of modern sensing systems, referred to as sensor management, is also a topic of significant interest currently, with systems attempting to obtain better measurements and improve performance towards a sensing task. This paper considers the problem of multiagent sensor management for searching and tracking of a single target in an urban environment.

Approaches to sensor management for searching and tracking typically adopt random finite set (RFS) based tracking algorithms which separate the multitarget state into disjoint subsets of tracked and untracked targets, such as the probability hypothesis density (PHD) filter [11], generalised la-

belled multi-Bernoulli (GLMB) filter [12], [13] or Poisson multi-Bernoulli mixture (PMBM) filter [14]–[16]. With formal representation of undetected targets, these filters allow for the adoption of popular information theoretic objective functions in a partially observable Markov decision process (POMDP). LeGrand et al presents a variation on the GLMB with the cell multi-Bernoulli filter for searching while tracking sensor management with a single sensing agent [17], [18]. The PMBM is adopted by [19] for searching and tracking with a non-myopic formulation. Non-myopic approaches are also adopted in [20], [21], with both implementing the PHD filter.

In the urban setting, literature considering road constrained targets focusses on incorporating this information into the tracking algorithm. To this end, single target tracking using a particle filter approach is presented in [22] which was extended to the Gaussian mixture cardinalised PHD filter for multiple targets [23]. Multiple model techniques for road network informed tracking are also presented in [24]. Ramirez et al present a target searching algorithm adopting particle filter based target tracking and entropy based guidance of a mobile sensor [25]. The work was later extended with the addition of static sensors and using road network information to constrain target and sensor motion [26].

An alternative to the common information theoretic algorithms mentioned so far is coverage control. This is a popular approach for performing surveillance, with multiple sensing agents, from the control community [27], [28]. The approach designs control actions for multiple sensors performing distributed sensing tasks, [29], and has received much attention with applications in source term estimation [30], agriculture [31] and surveillance [32]. Most works in this area however focus little attention on the object or density that is to be covered by the sensing agents.

This work aims to bring the coverage control objective into the searching and tracking sensor management POMDP. Through adoption of a dynamic coverage control objective [33], search specific tracking algorithms, such as those mentioned above, are not required, with many other filters able to produce the underlying coverage density.

We present a coverage control inspired objective for a multiagent searching and tracking problem. The coverage objective is implemented in a Monte Carlo tree search (MCTS) framework for a non-myopic implementation and alleviation of differentiability requirements traditionally required for coverage control laws. The visibility informed Bernoulli filter [34] is adopted to handle limited field of view (FOV) sensors exhibiting missed and false target detections and to leverage

^{*}Authors are with the Department of Aeronautical and Automotive Engineering, Loughborough University, UK. Email: {t.glover, j.a.c.knowles, m.j.coombes}@lboro.ac.uk

[†]Authors are with the Defence Science Technology Laboratory, Salisbury, UK. Email: stonessoup@dstl.gov.uk

This work was supported the Defence Science Technology Laboratory (Dstl), part of the UK Ministry of Defence (MOD), through the Sensor Fusion and Management (SFM) project, under Contract ACC6052263. Authors would like to thank Dale Richards of Thales UK for his contributions.

target existence probability estimation as an indication of whether a target is tracked. Available road network information is incorporated into the sensor management objective, creating environment tailored searching and tracking behaviour. Presented contributions are implemented within the Stone Soup open source tracking and state estimation Python framework [35], allowing the community to take advantage of the algorithm flexibility with the already established tracking and sensor management components within the framework. Simulated Urban Mobility (SUMO) [36], an open source traffic simulation tool, is also used to design a simulation scenario based on a real city environment, providing road network information and target ground truth trajectory.

The remainder of this paper is organised as follows. Sec. II formally defines the searching and tracking problem and sensor management objective. Sec. III and Sec. IV present the target tracking and sensor management algorithms. Details of the implementation within the Stone Soup framework are provided in Sec. V. Simulation scenario and algorithm parameters are introduced in Sec. VI, before presenting the results and discussion of algorithm performance in Sec. VII. The paper is concluded and future work discussed in Sec. VIII

II. PROBLEM FORMULATION

This work considers the problem of searching for and tracking a target as it moves through an urban environment. The environment is denoted $\mathcal{D}$ and contains sensing agents.

A. Target Kinematic Model

The target state, $\mathbf{x} \in \mathbb{R}^4$, is described by position and velocity in Cartesian space such that, at time $k-1$,

$$\mathbf{x}_{k-1} = [x_{k-1} \quad \dot{x}_{k-1} \quad y_{k-1} \quad \dot{y}_{k-1}]^T. \quad (1)$$

Target motion is assumed to follow legal requirements of a vehicle travelling in an urban environment, such as driving on the road network (denoted by $\mathcal{R}$), following speed limits and stopping when required at traffic signals. However, vehicle motion is assumed to approximate to a nearly constant velocity model for target tracking purposes. This is not a close approximation but this work is not focussed on modelling urban vehicle motion. Therefore, a nearly constant velocity model is defined with transition density given by

$$\pi(\mathbf{x}_k | \mathbf{x}_{k-1}) = \mathcal{N}(\mathbf{x}_k; \mathbf{F}_k \mathbf{x}_{k-1}, \mathbf{Q}_k), \quad (2)$$

where $\mathcal{N}(\cdot)$ represents a Gaussian and the state transition and process noise matrices are given by

$$\mathbf{F}_k = \mathbf{F} = \mathbf{I}_2 \otimes \begin{bmatrix} 1 & T \\ 0 & 1 \end{bmatrix} \quad \mathbf{Q}_k = \mathbf{Q} = q \cdot \mathbf{I}_2 \otimes \begin{bmatrix} \frac{T^3}{3} & \frac{T^2}{2} \\ \frac{T^2}{2} & T \end{bmatrix}. \quad (3)$$

$\mathbf{I}_a$ denotes the $a \times a$ identity matrix, T is the sample time between $k-1$ and k , $\otimes$ is the Kronecker product and q is the process noise intensity. Note the removal of time subscript k since $\mathbf{F}$ and $\mathbf{Q}$ are time invariant with constant T .

Since the target is unlikely to be tracked continuously from $k=0$, a Markov chain is introduced to model the probabilities

of transition between target presence and absence. Presence and absence in this case are referring to a binary random variable, when probability of transitioning from absence to presence (birth) and continued presence (survival) are given by the probability of birth, p_b and probability of survival, p_s , respectively. The Markov chain here is used to describe the probabilities of transitioning between presence and absence according to the transition probability matrix,

$$\Pi = \begin{bmatrix} (1-p_b) & p_b \\ (1-p_s) & p_s \end{bmatrix}. \quad (4)$$

B. Sensing Agent Model

Performing the searching and tracking task are n mobile sensor platforms, denoted by the set $\mathcal{A} = \{\mathcal{A}_i | i \in \mathcal{I} = \{1, 2, \dots, n\}\}$. $\mathcal{I}$ denotes the index set of the agents, containing the indices of each agent. The state of agent i at time k , $\mathbf{s}_k^i \in \mathbb{R}^2$, denotes position in Cartesian space, $\mathbf{s}_k^i = [x_k^i \quad y_k^i]^T$ and the set of all agent positions is $\mathbf{S}_k = \{\mathbf{s}_k^i | i \in \mathcal{I}\}$.

Sensor agent motion takes the form of position changes, with the motion model given by

$$\mathbf{s}_{k+1}^i = \mathbf{s}_k^i + \mathbf{u}_k^i = \mathbf{s}_k^i + [\delta_x^i \quad \delta_y^i]^T \quad (5)$$

where $\mathbf{u}_k^i$ is the control action applied to agent i and $\mathbf{u}_k^i \in \mathcal{U}_k^i$ where $\mathcal{U}_k^i$ denotes the feasible control action set. δ_x, δ_y are changes to xy components of the agent position. The feasible set confines agent motion to $\mathcal{D}$ and, at each time step, to a circle of radius $\rho_{\max}$ around the current position, leading to

$$\mathcal{U}_k^i = \left\{ \begin{bmatrix} \rho^i \cdot \sin(\phi) \\ \rho^i \cdot \cos(\phi) \end{bmatrix} \middle| \rho \in [0, \rho_{\max}^i], \phi^i \in [0, 2\pi], \mathbf{s}_{k+1}^i \in \mathcal{D} \right\}. \quad (6)$$

The set containing actions for all agents is given by $\mathbf{U}_k = \{\mathbf{u}_k^i | i \in \mathcal{I}\}$

C. Sensor Model

Each agent is equipped with a sensor capable of returning range-bearing measurements of the target when within an FOV, denoted by D_k^i , which is formed by bearing space, $F_\theta^i \in (0, 2\pi]$, range space, $F_r^i = [0, r_{\max})$ and agent position at time k . The overall detectable set is given by

$$D_k = \{D_k^i | i \in \mathcal{I}\}. \quad (7)$$

When the target state is within D_k , a measurement of the target will be returned according the detection probability $p_D(\mathbf{x}_k)$, given by

$$p_D(\mathbf{x}_k) = \begin{cases} p_D(\cdot) & \text{if } \mathbf{x}_k \in D_k \\ 0 & \text{otherwise,} \end{cases} \quad (8)$$

which assumes constant detection probability within the FOV.

The range-bearing measurement equation is given by

$$h^i(\mathbf{x}_k) = \begin{bmatrix} \|\mathbf{x}_{xy,k} - \mathbf{s}_k^i\| \\ \text{atan2}\left(\frac{x_k - x_k^i}{y_k - y_k^i}\right) \end{bmatrix}, \quad (9)$$

where $\mathbf{x}_{xy}$ is the xy component of the target state. The modelled sensor is assumed to return measurements corrupted with Gaussian noise, missed and false detections. Each sensor agent may return multiple or no measurements at any timestep. Therefore, the measurement set is defined as $\mathbf{Z}_k^i = \{\mathbf{z}_{k,1}^i, \mathbf{z}_{k,2}^i, \dots, \mathbf{z}_{k,m_k}^i\}$ where $\mathbf{z}_{k,a}^i$ is the a^{th} measurement vector and m_k^i denotes the number of measurements returned by agent i at time k . The construction of the measurement set therefore follows

$$\mathbf{Z}_k^i = \begin{cases} \{h^i(\mathbf{x}_k) + \omega_k^i\} \cup \mathbf{C} & \text{if } \mathbf{x}_k \in D_k^i \text{ and } \xi < p_D^i(\cdot) \\ \emptyset \cup \mathbf{C} & \text{otherwise,} \end{cases} \quad (10)$$

where $\mathbf{C}$ is a set of false detections, following a distribution $c(\mathbf{z})$ and Poisson cardinality distribution of constant mean, λ . ω_k^i is zero mean Gaussian noise with covariance $\mathbf{R}^i = \begin{bmatrix} \sigma_r^2 & 0 \\ 0 & \sigma_\theta^2 \end{bmatrix}$ and ξ is a uniform random number enforcing detection probability.

The conventional measurement likelihood function follows a typical Gaussian according to

$$g_k^i(\mathbf{z}|\mathbf{x}_k) = \mathcal{N}(\mathbf{z}; h^i(\mathbf{x}_k), \mathbf{R}^i), \quad (11)$$

for agent i . Later uses of this function omit the agent superscript for brevity, but its inclusion is implied when evaluating measurements in the overall set $\mathbf{Z}_k = \{\mathbf{Z}_k^i | i \in \mathcal{I}\}$.

D. Objective

This work adopts a coverage control objective focussed on acquiring information about a density according to a defined information reference. We adopt the idea presented by Hübel et al in this searching and tracking scenario [33]. The information map, $I_k(\mathbf{p}, \mathbf{S}_k)$, allocates Cartesian xy points in the environment, $\mathbf{p} \in \mathcal{D}$, a level of information based on sensing actions from the agents. This information is unitless and $I_k(\mathbf{p}, \mathbf{S}_k) \rightarrow \mathbb{R}^+$. The information map evolves with time according to

$$I_{k+1}(\mathbf{p}, \mathbf{S}_{k+1}) = \Delta I_k(\mathbf{p}, \mathbf{S}_k) + \mathcal{M}_{k+1}(\mathbf{p}, \mathbf{S}_{k+1}) \quad (12)$$

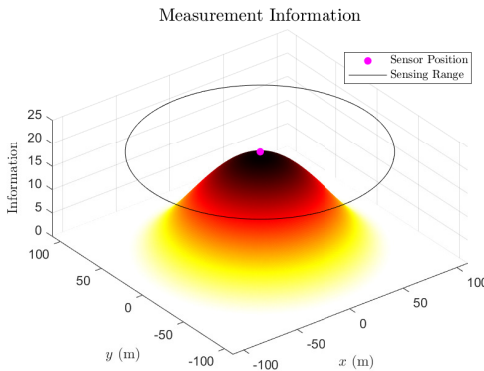


Fig. 1: Measurement information illustration with $\mathcal{M}_{\max} = 20$ and $r_{\max} = 100$ m.

where $\mathcal{M}_{k+1}(\mathbf{p}, \mathbf{S}_{k+1})$ quantifies the new information from all agents between k and $k+1$ and constant Δ decays the current state of information ($0 \leq \Delta \leq 1$).

Measurement information is modelled by a quadratic expression within D , similar to [33], reflecting the degradation of information quality with range. This is given by

$$\mathcal{M}_{k+1}(\mathbf{p}, \mathbf{S}_{k+1}) = \sum_{i \in \mathcal{I}} \begin{cases} \frac{\mathcal{M}_{\max}}{(\kappa \cdot r_{\max})^4} (d_{k+1}^{i,2} - (\kappa \cdot r_{\max})^2)^2, \\ \text{if } d_{k+1}^i \leq (\kappa \cdot r_{\max}) \text{ and } \mathbf{p} \in D_{k+1}^i \\ 0 & \text{otherwise,} \end{cases} \quad (13)$$

where $\mathcal{M}_{\max}$ is the maximum information gained by a sensor, $\kappa \leq 1$ is a constant defining the percentage of the sensor range to use and d_{k+1}^i is the distance from s_{k+1}^i to all $\mathbf{p}$. An illustration of this for a single sensor is provided in Fig. 1.

With the information map and model introduced, the reward for this problem is presented as

$$r(b(\mathbf{x})) = - \int_{\mathcal{D}} e(I_{\text{ref}}(\mathbf{p}) - I(\mathbf{p}, \mathbf{S})) \cdot \psi(\mathbf{p}, b(\mathbf{x})) d\mathbf{p}, \quad (14)$$

where $e(\cdot)$ calculates the error between information map and an information reference, $I_{\text{ref}}(\mathbf{p})$, $\psi(\cdot)$ is a function describing the underlying probability density in relation to which the information error is to be minimised, introduced in Sec. IV-C. $b(\mathbf{x})$ is the posterior target belief given by $b_{k+1}(\mathbf{x}) \triangleq p(\mathbf{x}_{k+1} | \mathbf{Z}_{1:k+1}, \mathbf{U}_{1:k})$ at $k+1$ for example.

With this reward, the action value function for the problem is defined as

$$Q^{\varpi}(b_{k+1}(\mathbf{x}), \mathbf{U}_k) = \hat{\mathbf{Z}}_{k+1}^{\mathbb{E}} \left[r_{k+1}(b_{k+1}(\mathbf{x})) + \gamma \cdot V^{\varpi}(b_{k+1}(\mathbf{x})) \middle| b_k(\mathbf{x}), \mathbf{U}_k \right], \quad (15)$$

where $\hat{\mathbf{Z}}$ is the future measurement set, ϖ is the action policy for all agents, γ is the discount factor on future rewards and $V^{\varpi}(b_{k+1})$ is the value function. This is given by

$$V^{\varpi}(b_{k+1}(\mathbf{x})) = \hat{\mathbf{Z}}_{k+2:k+H}^{\mathbb{E}} \left[\sum_{h=k+2}^{k+H} \gamma^{h-k+2} \cdot r(b_h(\mathbf{x}), \varpi) \middle| b_k(\mathbf{x}), \varpi \right], \quad (16)$$

which is the discounted sum of rewards over horizon H . Therefore, the overall objective for this problem is to obtain the next action set for the sensing agents according to

$$\mathbf{U}_k \in \underset{\mathbf{U}_k \in \mathcal{U}}{\operatorname{argmax}} [Q^{\varpi}(b_{k+1}(\mathbf{x}), \mathbf{U}_k)] \quad (17)$$

III. TARGET TRACKING

Target tracking is performed by the visibility informed Bernoulli filter, which will both estimate if the target is present and estimate its state [34]. This filter is an ideal choice for the presented problem, with it being the optimal sequential Bayesian estimator for single target tracking in the presence of missed and false detections and target appearance/disappearance [37]. The visibility informed variant is adopted to model the limited FOV sensor, similar to [38],

as will be presented below. The filter introduced here operates centrally, on the set of measurements from all agents, $\mathbf{Z}_k$.

The filter models the multiobject posterior probability density function (PDF) as a Bernoulli RFS. This can be empty with probability $1 - q_{k|k}$ or populated with a single state estimate $\mathbf{x}$ with probability $q_{k|k}$ and the target belief PDF $b_{k|k}(\mathbf{x})$. With this model, the PDF is entirely represented by the probability of existence $q_{k|k}$ and belief $b_{k|k}(\mathbf{x})$.

The objective of the Bernoulli filter is to sequentially propagate $q_{k|k}$ and $b_{k|k}(\mathbf{x})$ through time. This is done with prediction and update steps, in the typical Bayesian approach. From Mahler [39], who refers to the Bernoulli filter as the joint target-detection and tracking (JOTT) filter, the prediction equations are given by

$$q_{k|k-1} = p_b(1 - q_{k-1|k-1}) + p_s q_{k-1|k-1} \quad (18)$$

and

$$b_{k|k-1}(\mathbf{x}) = \frac{p_b(1 - q_{k-1|k-1})\mathcal{E}_{k|k-1}(\mathbf{x})}{q_{k|k-1}} + \frac{p_s q_{k-1|k-1} \int \pi_{k|k-1}(\mathbf{x}|\mathbf{x}') b_{k-1|k-1}(\mathbf{x}') d\mathbf{x}'}{q_{k|k-1}}. \quad (19)$$

Here, $\mathbf{x}'$ compactly denotes the prior target state at $k-1$ and $\mathcal{E}_{k|k-1}(\mathbf{x})$ is the target “birth” density which will be discussed in Sec. III-A. If the detector measurement model is adopted [37], where at most one measurement is returned from the target, the update equations are given by

$$q_{k|k} = \frac{1 - \delta_k}{1 - q_{k|k-1}\delta_k} q_{k|k-1} \quad (20)$$

$$b_{k|k}(\mathbf{x}) = \frac{1 - p_D(\mathbf{x}) + p_D(\mathbf{x}) \sum_{\mathbf{z} \in \mathbf{Z}_k} \frac{g_k(\mathbf{z}|\mathbf{x})}{\lambda_C(\mathbf{z})}}{1 - \delta_k} b_{k|k-1}(\mathbf{x}), \quad (21)$$

with

$$\delta_k = \int p_D(\mathbf{x}) b_{k|k-1}(\mathbf{x}) d\mathbf{x} - \sum_{\mathbf{z} \in \mathbf{Z}_k} \frac{\int p_D(\mathbf{x}) g_k(\mathbf{z}|\mathbf{x}) b_{k|k-1}(\mathbf{x}) d\mathbf{x}}{\lambda_C(\mathbf{z})}. \quad (22)$$

A. Sequential Monte Carlo Approximation

Computational intractability of the set integral in (19), (21) and (22) is alleviated by approximating the target PDF with a set of N weighted particles defined as $\{w_k^{(j)}, \mathbf{x}_k^{(j)}\}_{j=1}^N$. The birth density is approximated with a second set of B weighted particles which introduces the approximated prior belief density as $b_{k|k-1}(\mathbf{x}) \approx \sum_{j=1}^{N+B} w_{k|k-1}^{(j)} \delta_{\mathbf{x}_{k|k-1}^{(j)}}(\mathbf{x})$, where $\delta_{\mathbf{a}}(\mathbf{x})$ is the Dirac delta function focussed at point $\mathbf{a}$. This approximation of the target belief density does not affect the probability of existence prediction and update meaning that (18) and (20) are implemented directly.

For the proposal distribution, the transition density is adopted for both surviving and birth particle sets. The birth density is constructed adaptively using the set of observations,

as introduced in [37]. Therefore, $\mathcal{E}_{k|k-1}(\mathbf{x})$ is approximated according to

$$\mathcal{E}_{k|k-1}(\mathbf{x}) \approx \int \pi_{k|k-1}(\mathbf{x}|\mathbf{x}') \mathcal{E}_{k-1}(\mathbf{x}'; \mathbf{Z}_{k-1}) d\mathbf{x}', \quad (23)$$

where $\mathcal{E}_{k-1}(\mathbf{x}'; \mathbf{Z}_{k-1})$ is constructed from an equally weighted Gaussian mixture with each component focussed at $\mathbf{z}_{k-1} \in \mathbf{Z}_{k-1}$, converted to Cartesian space according to the measurement model. If we let $\mathbf{x}_{k-1}^{(j)} \sim \mathcal{E}_{k-1}(\mathbf{x}; \mathbf{Z}_{k-1})$ for $j = N+1, \dots, N+B$, be a sample of the birth density at $k-1$, the particle weight prediction is defined as

$$\mathbf{x}_{k|k-1}^{(j)} \sim \mathcal{Q}_k(\mathbf{x}_k|\mathbf{x}_{k-1}^{(j)}, \mathbf{Z}_k), \quad j = 1, \dots, N, N+1, \dots, N+B \quad (24)$$

$$w_{k|k-1}^{(j)} = \begin{cases} \frac{p_s q_{k-1|k-1}}{q_{k|k-1}} \frac{\pi_{k|k-1}(\mathbf{x}_{k|k-1}^{(j)}|\mathbf{x}_{k-1}^{(j)})}{\mathcal{Q}_k(\mathbf{x}_{k|k-1}^{(j)}|\mathbf{x}_{k-1}^{(j)}, \mathbf{Z}_k)} \cdot w_{k-1}^{(j)}, & j = 1, \dots, N \\ \frac{p_b(1 - q_{k-1|k-1})}{q_{k|k-1}} \frac{\mathcal{E}_{k|k-1}(\mathbf{x}_{k|k-1}^{(j)}|\mathbf{x}_{k-1}^{(j)})}{\mathcal{Q}_k(\mathbf{x}_{k|k-1}^{(j)}|\mathbf{x}_{k-1}^{(j)}, \mathbf{Z}_k)} \frac{1}{B}, & j = N+1, \dots, N+B, \end{cases} \quad (25)$$

where $\mathcal{Q}_k(\mathbf{x}_k|\mathbf{x}_{k-1}^{(j)}, \mathbf{Z}_k)$ is the transition density based proposal. Through approximation of all integrals in update equations by summations, prior to normalisation and resampling, denoted by $\sim$ and $*$ respectively, δ_k and $w_{k|k}^{(j)}$ become

$$\delta_k \approx \sum_{j=1}^{N+B} p_D(\mathbf{x}_{k|k-1}^{(j)}) w_{k|k-1}^{(j)} - \sum_{\mathbf{z} \in \mathbf{Z}_k} \sum_{j=1}^{N+B} p_D(\mathbf{x}_{k|k-1}^{(j)}) \frac{g(\mathbf{z}|\mathbf{x}_{k|k-1}^{(j)})}{\lambda_C(\mathbf{z})} w_{k|k-1}^{(j)} \quad (26)$$

$$\tilde{w}_{k|k}^{(i,*)} = \left[1 - p_D(\mathbf{x}_{k|k-1}^{(j)}) + p_D(\mathbf{x}_{k|k-1}^{(j)}) \sum_{\mathbf{z} \in \mathbf{Z}_k} \frac{g(\mathbf{z}|\mathbf{x}_{k|k-1}^{(j)})}{\lambda_C(\mathbf{z})} \right] w_{k|k-1}^{(j)}. \quad (27)$$

Following the weight update step above, systematic resampling is performed to downsample $N+B$ particles to the set of N surviving samples. Since resampling at this frequency may cause particle degeneracy, the Markov chain Monte Carlo (MCMC) move step is implemented for regularisation with Metropolis-Hastings acceptance criteria [24].

Two modifications from the visibility informed Bernoulli particle filter (VIBPF) are implemented here to handle the limited sensor FOV. This modifies the probability of detection and birth density for particles outside of D_k according to

$$p_D(\mathbf{x}_{k|k-1}^{(i)}) = \begin{cases} p_D(\cdot) & \text{if } \mathbf{x}_{k|k-1}^{(j)} \in D_k \\ \epsilon_{p_D} & \text{otherwise,} \end{cases} \quad \text{for } j = 1, \dots, N+B \quad (28)$$

and

$$\mathcal{E}_{k|k-1}(\mathbf{x}_{k|k-1}^{(j)}|\mathbf{x}_{k-1}^{(j)}) = \begin{cases} \pi_{k|k-1}(\cdot) & \text{if } \mathbf{x}_{k|k-1}^{(j)} \in D_k \\ \epsilon_{\mathcal{E}} & \text{otherwise} \end{cases} \quad (29)$$

for $j = N+1, \dots, N+B$.

Presentation of the affect that these modifications have on the Bernoulli particle filter is presented in [34].

IV. SENSOR PLANNING

Sensing actions are planned centrally for all agents by maximising the action value function in (17). This is done through the design of an MCTS algorithm which approximates (15) with iterative simulation runs. Before introducing this process, the design of information reference and future belief update technique are presented.

A. Information reference

Specifying the desired level of information points, $\mathbf{p}$, in the environment have, the information reference allows the objective in (14) to be tailored to the scenario at hand. In this work, the road network, $\mathcal{R}$, is used to inform the information reference since the target traverses on roads. Points closest to road network points, $\mathbf{p}_{RN} \in \mathcal{R}$, are allocated the highest information requirement (I_{RN}), which will decay quadratically to a uniform value (I_B) over range r_{RN} . This is given by

$$I_{\text{ref}}(\mathbf{p}) = \begin{cases} \frac{I_{RN}}{r_{RN}^4} (d_{RN}^2 - r_{RN}^2)^2 & \text{if } d_{RN} \leq r_{RN} \\ I_B & \text{if } d_{RN} > r_{RN}, \end{cases} \quad (30)$$

where

$$d_{RN} = \min [\|\mathbf{p} - \mathbf{p}_{RN}\| \mid \forall \mathbf{p}_{RN} \in \mathcal{R} \text{ and } \forall \mathbf{p} \in \mathcal{D}]. \quad (31)$$

specifies the minimum distance of $\mathbf{p}$ to any point on the road network. The result of this is shown in Fig. 2. Flexibility of this information reference allows the algorithm to be tailored to many scenarios by changing the above expression.

B. Future Belief Update

Without knowledge about the observations each sensing agent will receive in the future, some assumptions are made in order to predict the future target belief. A sequential importance sampling (SIS) particle filter is implemented for this process under the assumption of ideal measurements.

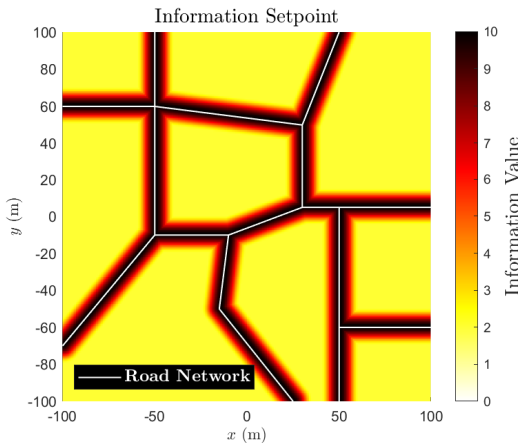


Fig. 2: Information reference example with road network. $I_{RN} = 10$, $I_B = 2$, $r_{RN} = 10$ m.

The first stage of this process is to enact the sensor agent transition model with a candidate action. New actions are generated according to a Monte Carlo policy, ϖ , such that

$$\mathbf{U}_k = \left\{ \mathbf{u}_k^i \mid \forall i \in \mathcal{I} \mid \rho^i \sim \rho_{\max} \cdot \sqrt{U[0, 1]} \text{ and } \phi^i \sim U[0, 2\pi] \right\}, \quad (32)$$

where ρ^i and ϕ^i are used to calculate $\mathbf{u}_k^i$ according to (6), and are sampled to ensure uniformity in the Cartesian space, hence the root applied in sampling ρ^i . Future sensor position is then calculated according to (5).

Current belief, $b_k(\mathbf{x})$, is provided to the sensor planning algorithm at the beginning of each sensor planning process, as the weighted particle set $b_k(\mathbf{x}) = \left\{ w_k^{(j)}, \mathbf{x}_k^{(j)} \right\}_{j=1}^N$. This particle set is drawn according the proposal distribution (transition density), given by

$$\mathbf{x}_{k+1|k}^{(j)} \sim \pi_{k+1|k} \left(\mathbf{x}_{k+1}^{(j)} \mid \mathbf{x}_k^{(j)} \right), \quad j = 1, \dots, N. \quad (33)$$

In seeking the future measurement, we generate a single measurement per sensor by using the mean target state. Therefore the predicted measurement set at $k+1$ is given by

$$\mathbf{Z}_{k+1} = \left\{ z_{k+1}^i = \begin{cases} h^i(\hat{\mathbf{x}}_{k+1|k}) & \text{if } \hat{\mathbf{x}}_{k+1|k} \in D_{k+1}^i \\ \emptyset & \text{otherwise} \end{cases} \mid \forall i \in \mathcal{I} \right\}, \quad (34)$$

where $\hat{\mathbf{x}}_{k+1|k} = \sum_{j=1}^N w_k^{(j)} \cdot \mathbf{x}_{k+1|k}^{(j)}$ is the mean predicted target state. Since $\mathbf{Z}_{k+1}$ may contain more than one measurement (if $\hat{\mathbf{x}}_{k+1|k} \in D_{k+1}^i$ for more than one agent), particle weights are updated according to

$$\tilde{w}_{k+1}^{(j)} = w_k^{(j)} \cdot \prod_{\mathbf{z}_{k+1} \in \mathbf{Z}_{k+1}} g(\mathbf{z}_{k+1} \mid \mathbf{x}_{k+1|k}^{(j)}), \quad (35)$$

followed by normalisation. Resampling and regularisation are found to be unnecessary for future belief prediction in this work but may be implemented in addition to the above process, if degeneracy and impoverishment are a problem with longer horizon planning. Furthermore, the above process is relevant for predicting the belief from any timestep, $k+n-1$ for example, to the next immediate timestep, $k+n$, within the horizon H .

Considering the existence estimated by the VIBPF, this is assumed to be time invariant during the sensor planning process, $q_{k+H} = q_{k+H-1} = \dots = q_{k+1} = q_k$. This assumption is justified since there is no reasonable way of predicting target birth or death during the planning process. This assumption could be changed if a model for future target birth and death is available.

C. Information Reward Density

In traditional information theoretic approaches to sensor management problems, the predicted future belief is now used to calculate information gain, or divergence, directly. With the novel information coverage reward presented here, an additional step is required to calculate (14).

The adopted coverage control objective is to collect information with respect to a probability density, $\psi(\mathbf{p}, b(\mathbf{x}))$. The multiplication in (14) allocates higher rewards to actions

increasing information where $\psi(\mathbf{p}, b(\mathbf{x}))$ is higher. This work uses the target belief for this purpose, but for multiplication in (14) to be valid, $I(\mathbf{p}, \mathbf{S})$, $I_{\text{ref}}(\mathbf{p})$ and $b(\mathbf{x})$ require the same support. To accomplish this, the belief particle weights are transferred to points $\mathbf{p}$ in a similar approach to mesh representation of particles from [40]. A particle set, whose states are positions $\mathbf{p} \in \mathcal{D}$, is introduced as

$$b(\mathbf{p}) = \{w^*, \mathbf{p}\}_{\mathbf{p} \in \mathcal{D}}. \quad (36)$$

To calculate weights, let

$$W_{\mathbf{p}} = \left\{ w^{(j)} \in b(\mathbf{x}) \mid \underset{j=1, \dots, N}{\operatorname{argmin}} \|\mathbf{x}_{xy}^{(j)} - \mathbf{p}\| \right\} \quad (37)$$

represent the set of weights from $b(\mathbf{x})$ whose particle position components, $\mathbf{x}_{xy}^{(j)}$, are closest to point $\mathbf{p}$. Particle $\mathbf{p}$ is then allocated a weight which is the sum of weights in $W_{\mathbf{p}}$ ($w^* = \sum_{w \in W_{\mathbf{p}}} w$). This process is performed for all $\mathbf{p} \in \mathcal{D}$.

With this modified belief, the coverage density is defined as

$$\psi(\mathbf{p}, b(\mathbf{x})) = \begin{cases} \frac{1}{2}[(1-\alpha) \cdot b(\mathbf{p}) + \alpha \cdot s(\mathbf{p})] & \text{if } q \geq q_{\text{thr}} \\ s(\mathbf{p}) & \text{if } q < q_{\text{thr}}, \end{cases} \quad (38)$$

where $s(\mathbf{p})$ is a uniform density across $\mathcal{D}$, α is a user defined weighting term controlling a mixture between $b(\mathbf{p})$ and $s(\mathbf{p})$ and q_{thr} is a threshold applied to the probability of existence. This allows for automatic switching between searching for the target and maintaining coverage of its belief when it is found. The uniform and target density mixture prevents rewards from becoming zero for sensing agents far from the target belief. This allows other agents to continue monitoring the environment once the target is found. This weighting is small in this work, since only a single target is considered

D. Monte Carlo Tree Search

Calculation and minimisation of the action value function is done through the MCTS algorithm. Simulations over horizon H are conducted to incrementally estimate $Q^{\varpi}(b_k(\mathbf{x}), \mathbf{U}_k)$ for each set of agent actions to take next. Therefore, (15) is approximated according to

$$Q^{\varpi}(b_k(\mathbf{x}), \mathbf{U}_k) \approx \frac{1}{M} \sum_{m=1}^M r_{k+1}(b_{k+1}(\mathbf{x})) + \gamma \cdot V^{\varpi}(b_{k+1}(\mathbf{x})), \quad (39)$$

where M is the number of times action set $\mathbf{U}_k$ is simulated. This is done by constructing a search tree where each node represents a belief and the branches represent actions. Belief nodes at level $k+n$ in the tree are denoted by $C(b_{k+n})$ where $b_{k+n} \equiv b_{k+n}(\mathbf{x})$, for ease of writing and clarity. Each belief node will also carry the reward, belief, current action value estimate and visit count $N(b_{k+n})$.

Simulation commence by descending the tree until reaching a node with unevaluated actions, $|C(b)| < n_{\mathbf{U}}$, with $|C(b)|$ indicating the number of child nodes and $n_{\mathbf{U}}$ the number of actions to sample. Nodes are selected in this descent according to the UCB1 algorithm,

$$\mathbf{U} \leftarrow \underset{\mathbf{U} \in C(b)}{\operatorname{argmax}} + c \sqrt{\frac{\log N(b)}{N(b\mathbf{U})}}, \quad (40)$$

Algorithm 1: Dynamic information coverage MCTS

```

1 procedure PLAN( $b$ )
2   for  $\_ \in 1 : m$  do
3     SIMULATE( $b, H$ )
4   end
5   return  $\underset{\mathbf{U}}{\operatorname{argmax}} Q(b\mathbf{U})$ 
6 end
7 procedure SIMULATE( $b, d$ )
8   if  $d = 0$  then
9     return 0
10  end
11  if  $|C(b)| < n_{\mathbf{U}}$  then
12    Generate new action:  $\mathbf{U} \leftarrow \text{NEXTACTION}(b)$ 
13    Append new action to search tree:  $C(b) \leftarrow C(b) \cup \mathbf{U}$ 
14  end
15  Select best child node,  $\mathbf{U}$ , according to (40)
16  if  $N(b\mathbf{U}) = 0$  then
17    Predict and update belief according to (33) to (35)  $b'$ 
18    Calculate sensing information,  $\mathcal{M}(\mathbf{p}, \mathbf{S})$ , according to (13)
19    Calculate information state,  $I(\mathbf{p}, \mathbf{S})$  according to (12)
20    Calculate reward:  $r$  according to (14)
21    Run rollout:  $total \leftarrow r + \gamma \cdot \text{ROLLOUT}(b', d-1)$ 
22  else
23     $b', r \leftarrow$  recall from  $C(b\mathbf{U})$ 
24     $total \leftarrow r + \gamma \cdot \text{SIMULATE}(b', d-1)$ 
25  end
26  Increment visit counts:  $N(b) \leftarrow N(b) + 1$ ,
    $N(b\mathbf{U}) \leftarrow N(b\mathbf{U}) + 1$ 
27  Update action value:  $Q(b\mathbf{U}) \leftarrow Q(b\mathbf{U}) + \frac{total - Q(b\mathbf{U})}{N(b\mathbf{U})}$ 
28  return  $total$ 
29 end

```

where $N(b\mathbf{U})$ is the number of visits to candidate nodes resulting from action $\mathbf{U}$ [41]. This balances exploitation of rewarding nodes and exploration of less visited nodes in the tree. Once a node is reached that has unevaluated actions, an available action is selected according to ϖ and subsequent belief and reward predicted. A policy rollout is then executed from the current timestep in the tree until $k+H$ with future rewards discounted and rollout actions discarded. Only initially generated actions remain in the search tree. The result of this evaluation is finally backpropagated through the tree, updating the action value estimates for each node back to the root. This process will repeat until a specified number of iterations is complete, where the next action set is selected through maximisation of the estimated action value. This process is presented in Algorithm 1 with further detail on MCTS available in the literature, notably [42] which Algorithm 1 is based on.

V. IMPLEMENTATION DETAILS

The presented algorithm is implemented in the Stone Soup¹ open source target tracking and state estimation framework, allowing the community to directly adopt, implement and develop the approach for their own problems [35]. Through contributions from the fields of target tracking and sensor

¹<https://stonesoup.readthedocs.io/en/latest/>

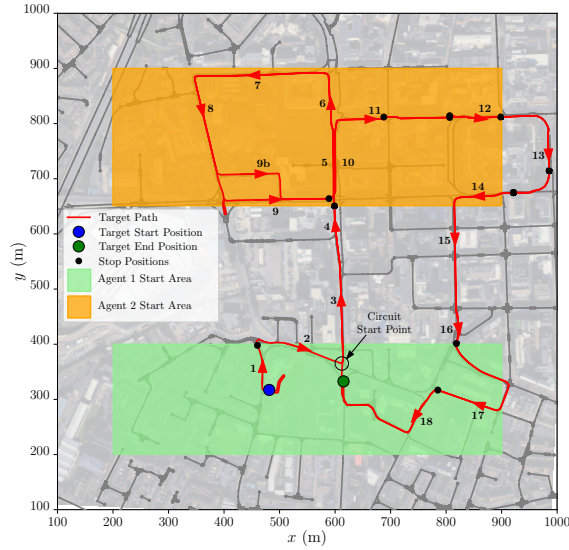


Fig. 3: An illustration of the target trajectory and sensor agent start locations for all three scenarios.

management, the Python toolkit is rapidly becoming one of the most advanced resources for such algorithms [43].

Generality and modularity has been maintained when contributing the following components to ensure their broad applicability either directly or with minimal further modification. The presented work has lead to the following new and developed components for the framework. Further information about Stone Soup can be found in its online documentation.

- *VisibilityInformedBernoulliParticlePredictor* and *VisibilityInformedBernoulliParticleUpdater* - New particle predictor and updater classes implementing the VIBPF [34].
- *InformationCoverageReward* - New reward class implementing the proposed information coverage reward function in (14) and reward density outlined in Sec. IV-C.
- *SUMOGroundTruthReader* - A reader class for extracting ground truth target trajectory from the Simulated Urban MObility open source traffic simulation package (SUMO) [36]. Support for road network extraction to implement (30) has also been included.
- *SamplePositionActionGenerator* - New base class for sampling-based position action generator types for actionable sensor platforms.
- *CircleSamplePositionActionGenerator* - Position action generator which samples uniformly within a circle around the current platform position, as described by (32).
- *CircleSampleActionableMovable* - Movable type implementing the above action generator.
- *MCTSRolloutSensorManager* - MCTS sensor management class has been improved upon the original form introduced in [44], by implementing the fixed horizon search which allows for a correct approximation of action value function (15) by (39).

VI. SCENARIO

A simulation scenario has been designed in order to demonstrate and analyse the algorithm. A visual representation of the scenario is provided in Fig. 3. This is designed using the

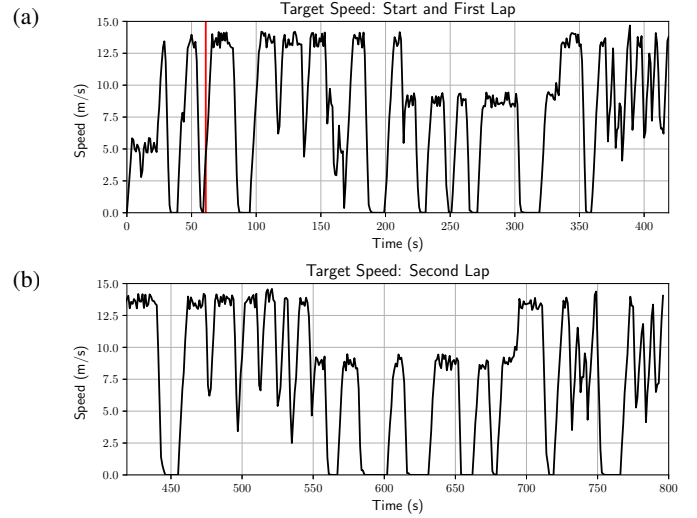


Fig. 4: Target speed profiles for (a) simulation start and first lap and (b) the second lap. The red line at 61 s in (a) indicates the start of the first lap.

SUMO software package and integrated with Stone Soup to extract the target trajectory, shown with the red line, and the road network shown in grey. Arrows along the path indicate the direction of travel for the target and numbers indicate the sequence. The point marked “Circuit Start Point” indicates the commencement of a circuit driven twice by the target. Laps are identical with exception to point 9b which is taken by the target on the second lap of the circuit. Black markers show points where the target is stationary at traffic signals. In total, the simulation duration is 797 s with $T = 1$ s.

Speed profile for the target is shown for the start and first lap in Fig. 4a and second lap in Fig. 4b. The red line in Fig. 4a indicates the circuit start point. It is clear from Fig. 4 that the target undergoes a number of acceleration, deceleration and almost constant velocity periods throughout the scenario, demonstrating the challenge that is faced by the filter, assuming nearly constant velocity.

The proposed algorithm is tested in this environment with two sensing agents, with sensing capabilities described in Sec. II, simulated across 200 Monte Carlo simulations with random start locations. Coloured regions in Fig. 3 indicate the areas where starting locations will be sampled.

Parameter	Value	Parameter	Value
Max range, $r_{\max}$	100 m	Detection probability, $p_D(\cdot)$	0.9
Range SD, σ_r	1 m	Clutter distribution, $c(\mathbf{z})$	$1/200\pi$
Bearing SD, σ_θ	$\pi/180$	Clutter rate, λ (per sensor)	1

TABLE I: Sensor Parameters

1) *Algorithm parameters:* All parameters for each aspect of the simulation are provided in Tables I to III. Each sensing agent is equipped with identical sensing capabilities according to Table I. These parameters are not intended to represent specific sensor capability, rather an agnostic omnidirectional range-bearing sensor, for generality. VIBPF parameters are provided in Table II and information model and MCTS pa-

Parameter	Value	Parameter	Value
Particle count, N, B	2000	Clutter distribution, $c(\mathbf{z})$	$1/200\pi$
Process noise intensity, q	10	Clutter rate, λ	1
Birth probability, p_b	0.02	Survival probability, p_s	0.98
Position prior, $\{\mathbf{x}_{xy,0}^{(j)}\}_{j=1}^N$	$U\left(\begin{bmatrix} 100 \\ 100 \end{bmatrix}, \begin{bmatrix} 1000 \\ 1000 \end{bmatrix}\right)$	Visibility modifications, $\epsilon_{pD}, \epsilon_\beta$	10^{-20}
Velocity prior, $\{\dot{\mathbf{x}}_{xy,0}^{(j)}\}_{j=1}^N$	$U\left(\begin{bmatrix} -10 \\ -10 \end{bmatrix}, \begin{bmatrix} 10 \\ 10 \end{bmatrix}\right)$	Prior existence probability, q_0	0.2

TABLE II: Tracking Parameters

Parameter	Value	Parameter	Value
Information Reward Parameters			
Information map precision	7.3 m ² , $ \mathbf{p} \in \mathcal{D} = 111556$	Maximum sensing information, $\mathcal{M}_{\max}$	24
Information decay, Δ	0.85	Effective sensor range κ	0.8
Base information reference, I_B	3	Road information reference, I_{RN}	53
Track existence threshold, q_{thr}	0.5	Road range, r_{RN}	20 m
		Search weight, α	10^{-6}
MCTS Parameters			
Horizon, H	3	Iteration count, m	400
Actions per timestep, $ \mathbf{U} $	196	Exploration factor, c	0.1
Maximum sensor travel, $\rho_{\max}$	20 m	Discount factor, γ	0.9

TABLE III: Planning Parameters

rameters in Table III. These parameters have been selected based on a sensitivity study for the presented scenario.

VII. RESULTS AND DISCUSSION

For illustration, Fig. 5 shows example sensor trajectories for a simulation run taken at 250 s. Background colour shows the current level of information from sensing actions. Algorithm performance is analysed according to several criteria. Optimal sub-pattern assignment (OSPA) is used to analyse target tracking performance with Euclidean distance cut off at $c_{\text{OSPA}} = 25$ m and exponent $p_{\text{OSPA}} = 1$ [45]. To penalise track continuity, if estimated probability of existence is less than $q_{\text{OSPA}} = 0.5$ then OSPA will assume no target is tracked and cardinality penalty applied. Existence estimation is the second performance criteria which will provide insight into tracking continuity as estimated by the VIBPF. The final four criteria aim to establish the algorithm search effectiveness and robustness of maintaining view of the target. These are the time to find the target, percentage of target trajectory within a sensor FOV, percentage of target states within a sensor FOV after the target is first found and the number of times the target was lost (outside of FOV for more than 10 seconds). All values are averaged across Monte Carlo simulations.

Figure 6 shows OSPA distance and existence estimation criteria with dashed blue line indicating mean, solid green line indicating the median and green shaded area indicating the

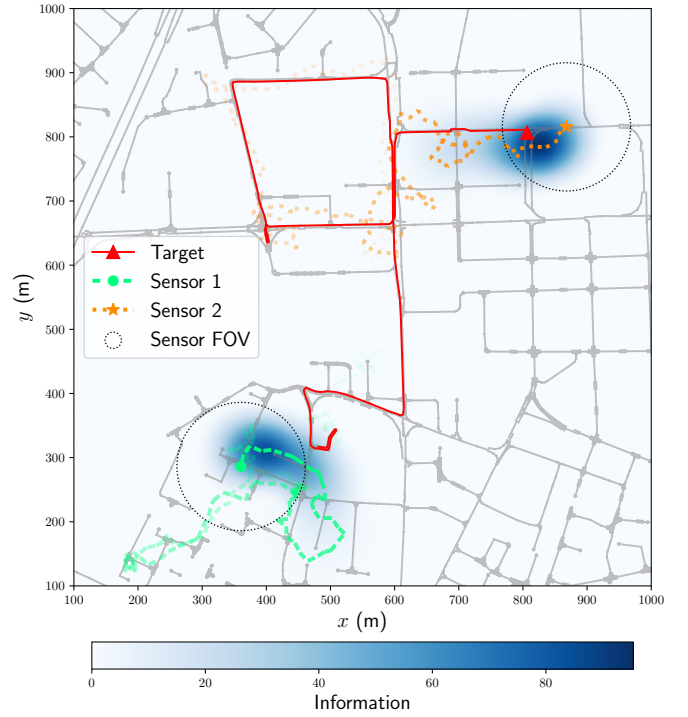


Fig. 5: An example overview of a simulation, showing both sensor positions, trajectories and FOVs; target position and trajectory and current information state, at 250 s.

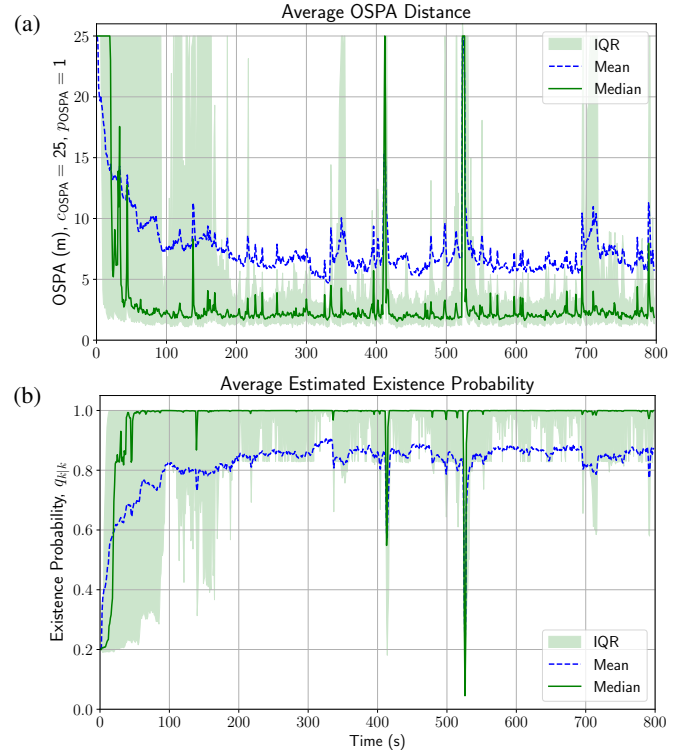


Fig. 6: Results for (a) OSPA distance and (b) probability of existence, with solid green line showing median value, dashed blue line showing mean value and shaded area indicating the IQR.

interquartile range (IQR). The remaining performance criteria are presented in Table IV. First considering OSPA, with an initially high value from unknown target location at $k = 0$, performance settles after 150 s, as indicated by the drop in IQR beyond this time. Mean OSPA settles to approximately 6.5 m while the median settles to approximately 2.0 m. This indicates that some worse performing simulations skew the mean, with the upper quartile suggesting that performance is more likely to be less than 5 m for 75 % of simulations after 150 s. Spikes in OSPA are present around 400 and 520 s. Examining the velocity profile in Fig. 4 at these times suggests that the erratic changes in speed may be the cause of these brief drops in performance, with this creating a large difference between predicted and actual target motion, likely resulting in loss of the target. Throughout both median and mean values are small spikes in OSPA. This may be caused by periods of acceleration and deceleration of the target where the implemented constant velocity model would react slowly to the changes.

Similar conclusions can be drawn from existence estimation in Fig. 6b. Starting at the prior value of 0.2, the existence probability increases over approximately 150 s until values settle, again highlighted by the IQR. Median existence estimation shows a faster convergence to 1.0, where it remains until brief performance dips at 400 and 520 s. Mean existence probability settles to approximately 0.8 here which aligns closely with the lower quartile. From the data, most simulations were able to achieve between 0.8 and 1.0 existence probability, with drops to 0.8 caused by only receiving measurements 90% of the time, despite the target being in view.

	Time to Find (s)	% Trajectory Tracked	% Trajectory Tracked after Found	Number of Target Losses
Mean	39.8	78.6	82.5	1.98
Median	13.0	87.7	93.5	2.0
IQR	47.0	27.4	23.6	2.0

TABLE IV: Searching and robustness performance criteria.

Finally considering criteria in Table IV, on average, the algorithm was able to find the target within 39.8 s with 50% of simulations achieving 13.0 s or less. With an IQR of 47.0 s, variance in the time to find the target is a direct result of the randomised sensor starting locations. Despite this, although not shown in Table IV, the algorithm was able to find the target in 100 % of Monte Carlo runs. Including the time to find the target, the algorithm covered close to 80% of its trajectory. Once found, the target remained in view for 82.5 % of the remaining target trajectory with 50 % of simulations achieving better than 93.5 %. Finally, the algorithm lost view of the target an average of 2 times per simulation, a likely correlation with spikes in Fig. 6 at 400 and 520 s.

Overall, the algorithm demonstrates good performance in both searching for and tracking the target when considering all performance criteria. The use of road network information in the information reference allows the algorithm to find the target in less than 5 % of the total simulation duration and

maintain coverage of the target for the remaining 82.5 %. Furthermore, considering the significant difference between the true and assumed target motion in the VIBPF, the algorithm is able to handle this adequately with good tracking performance considering environment scale at $8.1 \times 10^5 \text{ m}^2$.

VIII. CONCLUSION

This paper has presented a novel dynamic information coverage control objective for multiagent searching and tracking of a single target of interest in an urban environment. Implementation of the objective in an MCTS framework has created a non-myopic formulation that alleviates differentiability requirements of traditional coverage control approaches. Furthermore, the formulation allows for a diverse range of tracking algorithms to be implemented without the need for specifically searching and tracking oriented algorithms such as the PHD filter or other advanced RFS trackers. This flexibility is a feature that can be leveraged through its implementation within Stone Soup, a package equipped with a multitude of diverse tracking algorithms that could be adopted here. Despite this, the VIBPF is implemented for tracking in this work, for the convenience of the existence estimation allowing for switching between searching and tracking behaviours and handling sensor performance deficiencies.

Through simulation with realistic vehicle motion and real world road network data from SUMO and OpenStreetMaps, the algorithm demonstrates good performance, efficiently finding the target in less than 40 s and maintaining sight of the target for over 80 % of the simulation, on average. In fact, the algorithm found the target in 100 % of Monte Carlo runs.

There are a number of possible avenues to explore in future work. Implementing a more advanced motion model or a multiple model filter would better handle the complex target motion in an urban setting, as an apparent extension. It would be interesting to distribute the proposed algorithm to the sensing agents, preventing the combinatorial explosion of the action space with number of agents and potentially allowing the algorithm to be implemented in physical robotic experiments. It would also be interesting to consider the use of more limited sensing capability, such as range-only or bearing-only sensors, which creates a greater benefit for multiple agents to observe the same target. Extension of the algorithm to handle multi-target search-while-tracking would also be a valuable contribution, increasing the applicability of the proposed approach. Finally, it would be useful to establish the performance of this algorithm against existing searching and tracking algorithms; a straight forward task if they were to be contributed to Stone Soup.

REFERENCES

- [1] S. Gerasenko, A. Joshi, S. Rayaprolu, K. Ponnavaikko, and D. Agrawal, "Beacon signals: What, why, how, and where?" *Computer*, vol. 34, no. 10, pp. 108–110, Oct. 2001.
- [2] R. R. Murphy, "Search and rescue robotics," in *Springer Handbook of Robotics*, Springer Berlin Heidelberg, 2008.
- [3] L. Lin and M. A. Goodrich, "UAV intelligent path planning for wilderness search and rescue," in *2009 IEEE/RSJ International Conference on Intelligent Robots and Systems*, IEEE, 2009, pp. 709–714.

- [4] Y.-C. Du, M.-X. Zhang, H.-F. Ling, and Y.-J. Zheng, "Evolutionary planning of multi-UAV search for missing tourists," *IEEE Access*, vol. 7, pp. 73 480–73 492, 2019.
- [5] S. O. Murphy, C. Sreenan, and K. N. Brown, "Autonomous unmanned aerial vehicle for search and rescue using software defined radio," in *2019 IEEE 89th Vehicular Technology Conference (VTC2019-Spring)*, Kuala Lumpur, Malaysia: IEEE, Apr. 2019, pp. 1–6.
- [6] B. Thomas, J. D. Holland, and E. O. Minot, "Wildlife tracking technology options and cost considerations," *Wildlife Research*, vol. 38, no. 8, p. 653, 2011.
- [7] O. Cliff, R. Fitch, S. Sukkarieh, D. Saunders, and R. Heinsohn, "Online localization of radio-tagged wildlife with an autonomous aerial robot system," in *Robotics: Science and Systems XI*, Robotics: Science and Systems Foundation, Jul. 2015.
- [8] O. M. Cliff, D. L. Saunders, and R. Fitch, "Robotic ecology: Tracking small dynamic animals with an autonomous aerial vehicle," *Science Robotics*, vol. 3, no. 23, Oct. 2018.
- [9] R. W. Beard, T. W. McLain, D. B. Nelson, D. Kingston, and D. Johanson, "Decentralized cooperative aerial surveillance using fixed-wing miniature UAVs," *Proceedings of the IEEE*, vol. 94, no. 7, pp. 1306–1323, 2006.
- [10] W. Bentz and D. Panagou, "3D dynamic coverage and avoidance control in power-constrained UAV surveillance networks," in *2017 International Conference on Unmanned Aircraft Systems (ICUAS)*, Miami, FL, USA: IEEE, Jun. 2017, pp. 1–10.
- [11] R. Mahler, "Multitarget bayes filtering via first-order multitarget moments," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 39, no. 4, pp. 1152–1178, Oct. 2003.
- [12] B.-T. Vo and B.-N. Vo, "Labeled random finite sets and multi-object conjugate priors," *IEEE Transactions on Signal Processing*, vol. 61, no. 13, pp. 3460–3475, 2013.
- [13] B.-N. Vo, B.-T. Vo, and D. Phung, "Labeled random finite sets and the bayes multi-target tracking filter," *IEEE Transactions on Signal Processing*, vol. 62, no. 24, pp. 6554–6567, 2014.
- [14] J. L. Williams, "Marginal multi-Bernoulli filters: RFS derivation of MHT, JPDA and association-based Member," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 51, no. 3, pp. 1664–1687, Mar. 2015.
- [15] Á. F. García-Fernández, J. L. Williams, K. Granström, and L. Svensson, "Poisson multi-Bernoulli mixture filter: Direct derivation and implementation," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 54, no. 4, pp. 1883–1901, Aug. 2018.
- [16] K. Granström, L. Svensson, Y. Xia, J. Williams, and Á. F. García-Fernández, "Poisson multi-Bernoulli mixture trackers: Continuity through random finite sets of trajectories," *2018 21st International Conference on Information Fusion, FUSION 2018*, pp. 973–981, 2018.
- [17] K. A. LeGrand, P. Zhu, and S. Ferrari, "Cell multi-Bernoulli (cell-MB) sensor control for multi-object search-while-tracking (SWT)," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 6, pp. 7195–7207, Nov. 2022.
- [18] K. A. LeGrand, P. Zhu, and S. Ferrari, "A random finite set sensor control approach for vision-based multi-object search-while-tracking," in *2021 IEEE 24th International Conference on Information Fusion (FUSION)*, Sun City, South Africa: IEEE, Nov. 2021, pp. 1–8.
- [19] P. Boström-Rost, D. Axehill, and G. Hendeby, "Sensor management for search and track using the Poisson multi-Bernoulli mixture filter," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 57, no. 5, pp. 2771–2783, 2021.
- [20] B. Charrow, N. Michael, and V. Kumar, "Active control strategies for discovering and localizing devices with range-only sensors," in *Algorithmic Foundations of Robotics XI: Selected Contributions of the Eleventh International Workshop on the Algorithmic Foundations of Robotics*, 2015, pp. 55–71.
- [21] H. V. Nguyen, H. Rezatofighi, B.-N. Vo, and D. C. Ranasinghe, "Multi-objective multi-agent planning for jointly discovering and tracking mobile object," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 05, pp. 7227–7235, Apr. 2020.
- [22] M. Ulmke and W. Koch, "Road-map assisted ground moving target tracking," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 42, no. 4, pp. 1264–1274, 2006.
- [23] M. Ulmke, O. Erdinc, and P. Willett, "GMTI tracking via the Gaussian mixture cardinalized probability hypothesis density filter," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 46, no. 4, pp. 1821–1833, 2010.
- [24] B. Ristic, S. Arulampalam, and N. Gordon, *Beyond the Kalman Filter: Particle Filters for Tracking Applications*. Artech House, 2004.
- [25] J.-P. Ramirez, E. A. Doucette, J. W. Curtis, and N. Gans, "Moving target acquisition through state uncertainty minimization," in *2014 American Control Conference*, Portland, OR, USA: IEEE, Jun. 2014, pp. 3425–3430.
- [26] J.-P. Ramirez-Paredes, E. A. Doucette, J. W. Curtis, and N. R. Gans, "Urban target search and tracking using a UAV and unattended ground sensors," in *2015 American Control Conference (ACC)*, Chicago, IL, USA: IEEE, Jul. 2015, pp. 2401–2407.
- [27] J. Cortes, S. Martinez, T. Karatas, and F. Bullo, "Coverage control for mobile sensing networks: Variations on a theme," in *Mediterranean Conference on Control and Automation*, Lisbon, Portugal, 2002, pp. 9–13.
- [28] J. Cortes, S. Martinez, T. Karatas, and F. Bullo, "Coverage control for mobile sensing networks," *IEEE Transactions on Robotics and Automation*, vol. 20, no. 2, pp. 243–255, Apr. 2004.
- [29] B. Wang, *Coverage Control in Sensor Networks*. Springer Science & Business Media, 2010.
- [30] R. V. Nanavati, M. J. Coombes, and C. Liu, "Distributed multi-robot source term estimation with coverage control and information theoretic based coordination," *Information Fusion*, vol. 111, p. 102503, Nov. 2024.
- [31] A. Din, M. Y. Ismail, B. Shah, M. Babar, F. Ali, and S. U. Baig, "A deep reinforcement learning-based multi-agent area coverage control for smart agriculture," *Computers and Electrical Engineering*, vol. 101, p. 108089, Jul. 2022.
- [32] G. Mathew, A. Surana, and I. Mezic, "Uniform coverage control of mobile sensor networks for dynamic target detection," in *49th IEEE Conference on Decision and Control (CDC)*, Atlanta, GA, USA: IEEE, Dec. 2010, pp. 7292–7299.
- [33] N. Hübel, S. Hirche, A. Gusliardi, T. Hatanaka, M. Fujita, and O. Sawodny, "Coverage Control with Information Decay in Dynamic Environments," *IFAC Proceedings Volumes*, vol. 41, no. 2, pp. 4180–4185, 2008.
- [34] T. J. Glover, C. Liu, and W.-H. Chen, "Visibility informed Bernoulli filter for target tracking in cluttered environments," in *2022 IEEE 25th International Conference on Information Fusion, FUSION*, 2022.
- [35] P. A. Thomas, J. Barr, B. Balaji, and K. White, "An open source framework for tracking and state estimation ('stone soup')," in *Signal Processing, Sensor/Information Fusion, and Target Recognition XXVI*, I. Kadar, Ed., vol. 10200, 2017, p. 1020008.
- [36] P. A. Lopez, E. Wiessner, M. Behrisch, et al., "Microscopic traffic simulation using SUMO," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, Maui, HI: IEEE, Nov. 2018, pp. 2575–2582.
- [37] B. Ristic, B.-T. Vo, B.-N. Vo, and A. Farina, "A tutorial on Bernoulli filters: Theory, implementation and applications," *IEEE Transactions on Signal Processing*, vol. 61, no. 13, pp. 3406–3430, 2013.
- [38] T. J. Glover, C. Liu, and W.-h. Chen, "Dual control inspired active sensing for bearing-only target tracking," in *2023 IEEE 26th International Conference on Information Fusion, FUSION*, 2023, pp. 1–8.
- [39] R. Mahler, *Statistical Multisource Multitarget Information Fusion*. Artech House, 2007.
- [40] Y. Liu, M. Coombes, and C. Liu, "Mesh-based consensus distributed particle filtering for sensor networks," *IEEE Transactions on Signal and Information Processing over Networks*, vol. 9, pp. 346–356, 2023.
- [41] P. Auer and N. Cesa-Bianchi, "Finite-time analysis of the multiarmed bandit problem," *Machine Learning*, vol. 47, pp. 235–256, 2002.
- [42] Z. Sunberg and M. Kochenderfer, *Online algorithms for POMDPs with continuous state, action, and observation spaces*, Sep. 2018.
- [43] S. Hiscocks, J. Barr, N. Perree, et al., "Stone soup: No longer just an appetiser," in *2023 26th International Conference on Information Fusion (FUSION)*, Charleston, SC, USA: IEEE, Jun. 2023, pp. 1–8.
- [44] T. J. Glover, R. V. Nanavati, M. Coombes, et al., "A Monte Carlo tree search framework for autonomous source term estimation in Stone Soup," in *2024 27th International Conference on Information Fusion (FUSION)*, Venice, Italy: IEEE, Jul. 2024, pp. 1–8.
- [45] B. Ristic, B.-N. Vo, and D. Clark, "Performance evaluation of multi-target tracking using the OSPA metric," in *13th Conference on Information Fusion, FUSION 2010*, IEEE, 2010, pp. 1–7.